

01 / 09

Ajouter des agents ne fait pas une orchestre

J'ai vu des stacks où trois agents s'appellent en rond, chacun avec sa mémoire, aucun contrat de sortie. Ça ressemble à du multi-agent. Ça n'orchestre rien.

- Un agent de plus n'est pas un chef d'orchestre
- Sans ordre, l'état se perd entre les appels
- Le statut succès peut mentir jusqu'au bout



02 / 09

Le piège du chat entre agents

Cas fréquent : un agent recherche, un autre rédige, un troisième vérifie. Ils se passent du texte libre. Le deuxième reformule une erreur du premier. Le troisième valide le style, pas le fond.

- Handoff en prose, pas en schéma
- Personne n'est responsable du résultat final
- Chaque agent relit tout l'historique
- Les outils se chevauchent sans frontière
- On débogue un dialogue, plus un workflow



03 / 09

Orchestrer, c'est décider avant l'appel

L'orchestration, je la vois comme le plan : qui parle, dans quel ordre, avec quel état, et quand on s'arrête. Le modèle exécute une étape. Il ne choisit pas l'architecture à chaque token.

- Un graphe ou un pipeline, pas une conversation ouverte
- Un état partagé, versionné, lisible
- Des rôles avec un mandat étroit
- Une condition d'arrêt écrite
- Un humain ou un critique aux portes à risque



04 / 09

Choisissez le motif, pas le framework

Je commence souvent par trois motifs. Pipeline si l'ordre est fixe. Superviseur si une étape route vers un spécialiste. Graphe si des boucles et des reprises sont nécessaires. Le framework vient après.

- Pipeline : A puis B puis C, état en entrée et sortie
- Superviseur : un routeur, des spécialistes muets entre eux
- Graphe : nœuds, arêtes, reprise sur échec
- Évitez que tout le monde parle à tout le monde
- Changez de motif quand le cas le force



05 / 09

L'état est le vrai contrat

Si deux agents ne s'échangent qu'un paragraphe, vous orchestrez de la prose. Je préfère un état typé : but, preuves, décision, incertitude. Chaque nœud lit un sous-ensemble, écrit un champ, rien d'autre.

- Schéma stable, objets, pas un roman
- Champs d'entrée et de sortie nommés
- Interdit de réécrire le but en silence
- Garder les preuves, pas seulement la conclusion
- Versionner l'état pour rejouer un run



06 / 09

Aiguillez outils et modèles par nœud

Un nœud de tri n'a pas besoin du modèle le plus capable, ni de tous les outils. J'attache à chaque étape un modèle, un budget, une liste d'outils courte. Moins de surface, moins de dérive.

- Un nœud, un job, une poignée d'outils
- Petit modèle pour classer et extraire
- Modèle plus capable seulement sur le raisonnement
- Budget tokens et timeout par nœud
- Pas d'outil d'écriture sur un nœud de revue

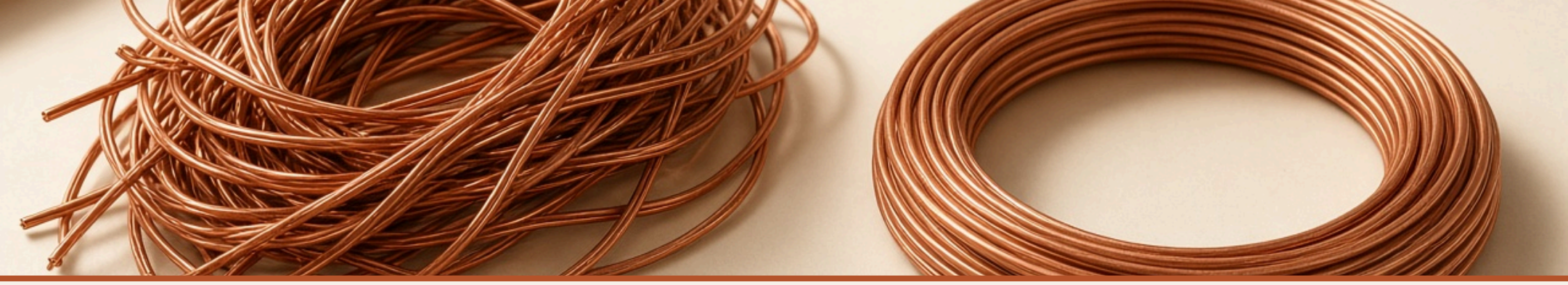


07 / 09

Tracez le run comme un workflow

Quand ça casse, je veux savoir quel nœud, quel état, quel outil, pourquoi la reprise. Un dump de prompts ne suffit pas. L'orchestration se debug comme un pipeline, pas comme un chat.

- Identifiant de run et de nœud
- Diff d'état avant et après
- Outil appelé, arguments, résultat
- Motif de reprise ou d'arrêt
- Replay possible à partir de l'état



08 / 09

Erreurs que je vois souvent

Elles tiennent rarement au modèle. Elles tiennent au fait qu'on a laissé les agents s'organiser tout seuls, sans partition écrite.

- Un agent chef qui refait tout le travail
- Aucun schéma entre les nœuds
- Boucle sans plafond de retries
- Revue avec les mêmes outils que la prod
- Observabilité limitée aux logs LLM



09 / 09

Qui tient la partition ?

Sur votre stack, l'orchestration est-elle un graphe écrit, ou une conversation qui s'invente à l'exécution ? L'étape utile : dessiner les nœuds, l'état, et la porte d'arrêt.

- Nommez le motif : pipeline, superviseur, graphe
- Écrivez le schéma d'état
- Décidez qui peut arrêter le run